HIGH LEVEL DESIGN
ON
IaC Provisioning for Real Estate System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Kushagra Gupta	EN22CS301546
02	Khushi Paliwal	EN2CS301505
03	Kritika Nagar	EN2CS301526
04	Kishna Sharma	EN2CS301510
05	Jayesh Jain	EN2CS301465
06	Kartik Gupta	EN22CS301488

Group Name: Group 07D5

Project Number: DO-07

Industry Mentor Name: Mr.Vaibhav

University Mentor Name: Prof.Ritesh Joshi

1. Introduction

The *IaC Provisioning for Real Estate System* is a DevOps-based automated platform designed to provision infrastructure and deploy applications efficiently using Infrastructure as Code (IaC).

The system eliminates manual configuration by automating infrastructure setup, application deployment, and scaling. It uses modern DevOps tools such as Terraform, Docker, and CI/CD pipelines to ensure faster, reliable, and consistent deployments.

1.1. Scope of the document

This document provides a high-level overview of the *IaC Provisioning for Real Estate System*, focusing on its architecture, components, and overall workflow.
It describes how the system automates infrastructure provisioning and application deployment using DevOps practices and tools such as Terraform, Docker, and CI/CD pipelines. The document outlines the major system components, their interactions, and the flow of data from development to deployment.
The scope of this document is limited to explaining the system design at a conceptual level. It does not include detailed implementation, configurations, or code-level specifications, which are covered in the Low-Level Design (LLD).
Overall, this document serves as a guide to understanding how the system is structured and how different components work together to deliver a scalable and automated solution.

1.2. Intended Audience

- **Project Evaluators:**
To assess the project's structure, design approach, and implementation of DevOps practices in solving real-world infrastructure and deployment challenges.

- **DevOps Engineers:**
To understand the system architecture, deployment workflow, and the use of tools such as Terraform, Docker, and CI/CD pipelines for infrastructure automation and application deployment.

- **System Architects:**
To analyze the overall system design, component interactions, scalability, and architectural decisions used in the project.

1.3. System overview

The *IaC Provisioning for Real Estate System* is a DevOps-based platform that automates infrastructure provisioning and application deployment. It uses Terraform to create cloud resources, Docker to containerize the application, and CI/CD pipelines to automate build and deployment.

When code is pushed to the repository, the pipeline is triggered, the application is built, and it is deployed on cloud infrastructure (AWS EC2). This ensures faster, scalable, and reliable deployment of the real estate application.

System Design

1.1. Application Design

The system is divided into four main layers:

- **Developer Layer:**
  Developers write application code and Terraform scripts and push them to the GitHub repository.
- **CI/CD Pipeline Layer:**
  GitHub Actions or Jenkins automatically triggers the pipeline to build, test, and deploy the application.
- **Containerization Layer:**
  Docker is used to create application containers, ensuring consistency across different environments.
- **Infrastructure Layer:**
  Terraform provisions AWS EC2 instances, and the application is deployed on the cloud infrastructure.

1.2. Process Flow

The high-level process flow is as follows:

1. Developer pushes code to GitHub
2. CI/CD pipeline is automatically triggered
3. Docker image is built
4. Terraform provisions infrastructure
5. EC2 instance is created and configured
6. Docker container is deployed
7. Application becomes live and accessible

1.3. Information Flow

The system follows a structured flow of data from development to deployment:

Developer → GitHub → CI/CD Pipeline → Docker → AWS EC2 → End User

- **Input:** Source code pushed by the developer
- **Processing:** Automated build, containerization, and deployment
- **Output:** Live application accessible to users

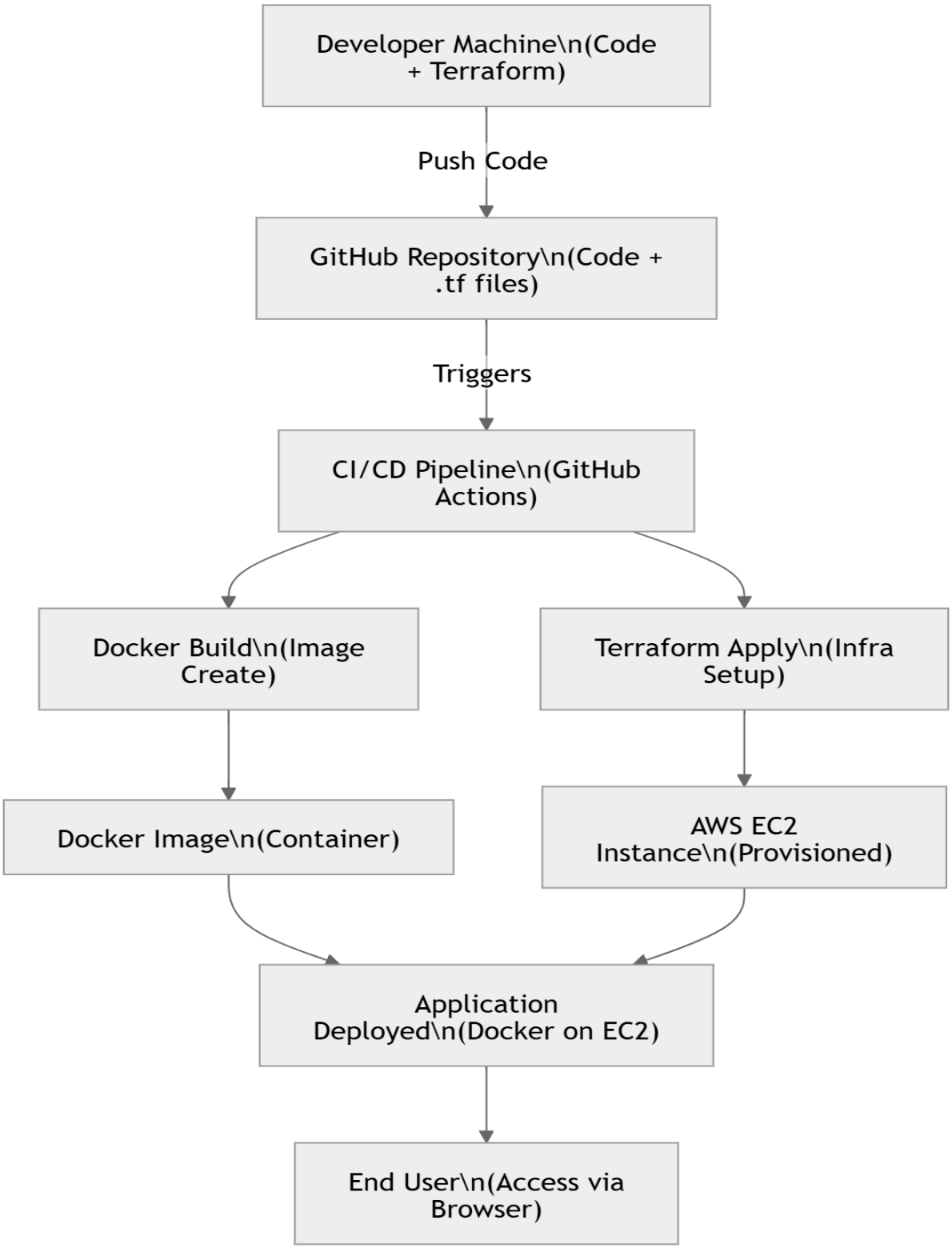

Figure 1. HIGH LEVEL DESIGN

Explanation -

1. Developer Layer

- The process starts with the **developer**, who writes application code and Infrastructure as Code (Terraform scripts).
- The code is pushed to a **GitHub repository**, which acts as version control.

---

2. CI/CD Pipeline

- Once the code is pushed, a **CI/CD pipeline (GitHub Actions or Jenkins)** is automatically triggered.
- The pipeline performs:
  - Code build
  - Testing
  - Deployment preparation

This ensures automation and reduces manual effort.

---

3. Docker & Containerization

- The application is packaged into a **Docker container**.
- Docker ensures that the application runs consistently across all environments (development, testing, production).

---

4. Infrastructure Provisioning (Terraform)

- **Terraform** is used to provision infrastructure on cloud platforms like AWS.
- It creates and configures resources such as **EC2 instances** automatically.

---

5. Deployment on Cloud (AWS EC2)

- The Docker container is deployed on the provisioned **EC2 instance**.
- The application becomes live and accessible over the internet.

---

6. End User Access

- Finally, **end users** access the application through a web browser using the public IP or domain.

---

7. Overall Workflow Summary

1. Developer writes and pushes code
2. GitHub triggers CI/CD pipeline
3. Docker builds application container
4. Terraform provisions infrastructure
5. Application is deployed on AWS EC2
6. Users access the live application

1.4. Components Design

The system consists of the following major components:

- **GitHub (Version Control):**
  Stores the application code, Terraform scripts, and configuration files.
- **CI/CD Pipeline (GitHub Actions / Jenkins):**
  Automates the build, test, and deployment process. It ensures continuous integration and continuous delivery of the application.
- **Docker (Containerization):**
  Builds the application image and packages it into a container. It provides a consistent runtime environment across development and production.
- **Terraform (Infrastructure as Code):**
  Automates the provisioning and configuration of infrastructure such as AWS EC2 instances using code.
- **AWS EC2 (Cloud Infrastructure):**
  Provides virtual machines where the application is deployed and executed.
- **Application (Real Estate System):**
  The main system that includes features such as user authentication and property listing, accessible to end users.

1.5. Key Design Considerations

- **Automation:** Reduces manual effort using IaC and CI/CD
- **Scalability:** Can handle increasing users by scaling infrastructure
- **Reliability:** Automated deployment reduces errors
- **Portability:** Docker ensures consistency across environments
- **Security:** Uses authentication and secure access (SSH keys)
- **Maintainability:** Infrastructure managed through code

1.6. API Catalogue

The system provides the following key APIs:

- **GET /** Loads the real estate application homepage.
- **POST /register** Allows users to create a new account.
- **GET /login** Authenticates users and provides secure access to the system.
- **GET /properties** Retrieves available property listings for users.

2. Data Design

This section describes how data is structured, stored, and managed within the *IaC Provisioning for Real Estate System*.

The system primarily handles two types of data:

1. **Application Data** (users, properties, transactions)
2. **Infrastructure Data** (Terraform state and configurations)

2.1. Data Model

The The system uses a database (such as MongoDB or MySQL) to store and manage application data in a structured format.

The main data entities include:

- **User Entity:**
  Stores user-related information such as user ID, name, email, and password. This data is used for authentication and account management.
- **Property Entity:**
  Contains details of real estate properties, including property ID, price, location, description, and other relevant attributes.
- **Transaction Entity:**
  Maintains records of property bookings or purchases made by users. It includes transaction ID, user ID, property ID, and transaction details.

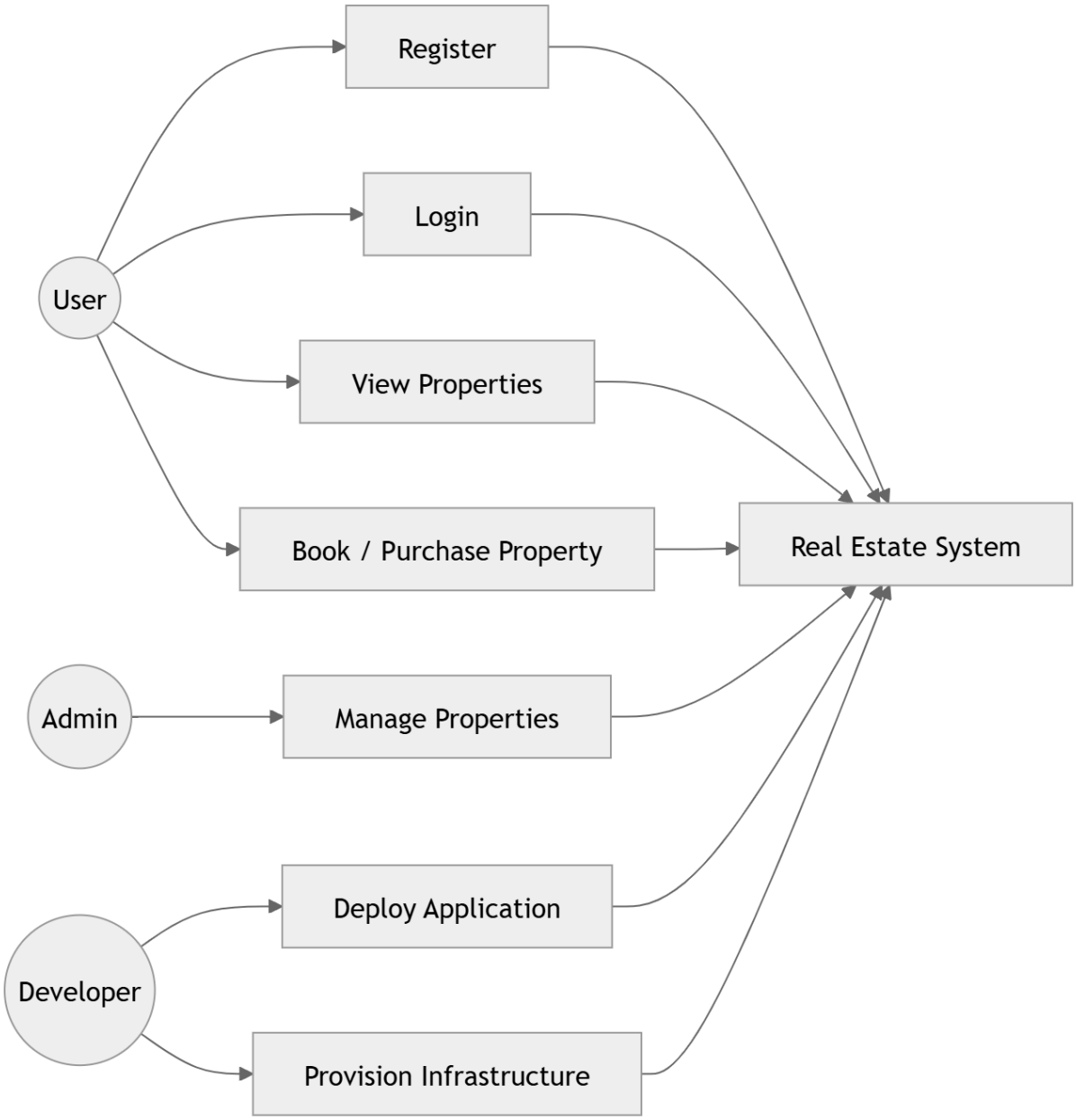

Figure 2: USE CASE Diagram

The Use Case Diagram represents how different actors interact with the *IaC Provisioning for Real Estate System*. It provides a high-level view of system functionality and shows the roles, responsibilities, and interactions of each actor.

---

1. Actors in the System

1.1 User (End User)
The user is the primary actor who interacts with the application to access real estate services.

Responsibilities:

- Register a new account
- Log into the system

- View available properties
- Book or purchase properties

1.2 Admin

The admin manages and controls system operations and data.

Responsibilities:

- Add new property listings
- Update property details
- Delete or manage properties
- Monitor system activity

1.3 Developer

The developer is responsible for system deployment and infrastructure management.

Responsibilities:

- Deploy application using CI/CD pipeline
- Provision infrastructure using Terraform
- Maintain system performance and updates

---

2. Use Cases Description

2.1 Register

- New users create an account
- User provides basic details (name, email, password)
- Data is stored in the database

2.2 Login

- Users authenticate using credentials
- System verifies user data
- Access is granted upon successful login

2.3 View Properties

- Users browse property listings
- System fetches property data from database
- Details such as price, location, and description are displayed

---

2.4 Book / Purchase Property

- User selects a property
- Initiates booking or purchase
- Transaction is recorded in the system

2.5 Manage Properties (Admin)

- Admin can:
- Ensures accurate and updated data

2.6 Deploy Application (Developer)

- Developer pushes code to GitHub
- CI/CD pipeline builds and deploys application

- Docker containers are used for deployment

2.7 Provision Infrastructure (Developer)

- Terraform scripts are executed
- AWS resources (EC2) are created
- Infrastructure is configured automatically

---

3. Interaction Flow

- The **User** interacts with frontend → sends requests to backend
- The **Admin** controls and updates system data
- The **Developer** ensures system deployment and infrastructure setup

All interactions are processed by the central system, which communicates with the database and infrastructure.

---

4. Importance of Use Case Diagram

- Provides clear understanding of system functionality
- Defines roles of different actors
- Helps in system planning and design
- Useful for communication between stakeholders

---

5. Conclusion

The Use Case Diagram gives a complete overview of how users, admins, and developers interact with the system. It highlights the key functionalities and ensures that all system operations are clearly defined and organized.

2.2. Data Access Mechanism

Data in the system is accessed through RESTful APIs, enabling smooth communication between different components.

- **Frontend:**
  Sends HTTP requests (such as GET, POST, PUT, DELETE) to interact with the system.
- **Backend:**
  Processes these requests using business logic and handles API routing.
- **Database:**
  Stores and retrieves data based on backend queries.

This architecture ensures efficient, secure, and seamless communication between the frontend, backend, and database.

2.3. Data Retention Policies

The system manages different types of data using specialized storage mechanisms:

- **Application Data:**
  Stored in a database such as MongoDB or MySQL, including user details, property information, and transactions.
- **Infrastructure Data:**
  Managed using Terraform state files, which keep track of provisioned resources and configurations.
- **Docker Images:**
  Stored in a container registry, enabling easy access and deployment of application containers.

2.4. Data Migration

Data migration refers to the process of transferring and updating data when there are changes in the system, such as database schema updates, infrastructure changes, or application upgrades.

In this system:

- Data migration may be required when modifying database structure (e.g., adding new fields or tables/collections).
- Migration scripts or tools can be used to update existing data without data loss.
- Terraform helps manage infrastructure changes without affecting application data.
- Backup mechanisms ensure that data can be restored in case of any failure during migration.

3. Interfaces

The system interacts with users and other components through the following interfaces:

- **User Interface (UI):**
  A web-based interface that allows users to access the real estate application, register, log in, and view property listings.
- **API Interface:**
  RESTful APIs are used for communication between the frontend and backend services, enabling data exchange and system functionality.
- **Command Line Interface (CLI):**
  Used by developers and DevOps engineers to run scripts, manage dependencies, and execute Terraform and Docker commands.
- **Database Interface:**
  The backend interacts with the database (MongoDB/MySQL) to store, retrieve, and manage application data efficiently.

4. State and Session Management

The system manages application state and user sessions to ensure a smooth and secure user experience.

- **State Management:**
  The application state includes user data, property listings, and transaction details. It is managed through the frontend (UI state) and backend database, ensuring that updated information is consistently available across the system.
- **Session Management:**
  User sessions are maintained after login to allow continuous interaction without repeated authentication. This can be implemented using session tokens such as JWT (JSON Web Tokens).
- **Session Persistence:**
  Session data (tokens) is stored on the client side (e.g., browser storage or cookies), allowing users to remain logged in across multiple requests.
- **Backend Validation:**
  Each request is validated on the backend using session tokens to ensure that only authorized users can access protected resources.

5. Caching

Caching is used to improve system performance and reduce response time by storing frequently accessed data.

- **API Response Caching:**
  Frequently requested data such as property listings can be cached to reduce repeated processing and improve response speed.
- **Database Query Caching:**
  Common database queries are cached to minimize database load and enhance performance.
- **Client-Side Caching:**
  Static resources such as images, CSS, and JavaScript files are stored in the browser cache to improve loading speed.

- **Performance Optimization:**
  Caching reduces latency, improves user experience, and ensures efficient use of system resources.

6. Functional Requirements

This section describes the core functionalities provided by the *IaC Provisioning for Real Estate System*.

6.1 User Authentication & Authorization

- Users can register and log in securely using email and password.
- Authentication ensures that only authorized users can access the system.

6.2 Infrastructure Provisioning

- The system automatically provisions infrastructure using Terraform.

- Resources such as AWS EC2 instances are created and configured without manual intervention.

6.3 CI/CD Automation

- The system integrates CI/CD pipelines (GitHub Actions/Jenkins).
- Automatically builds, tests, and deploys the application when code is pushed.

6.4 Application Deployment

- The application is containerized using Docker.
- Docker containers are deployed on cloud infrastructure (EC2).

6.5 Property Management

- Users can view property listings.
- The system manages property-related data efficiently.

6.6 System Monitoring & Control

- Basic monitoring ensures that infrastructure and application are running correctly.
- Logs and status checks help in maintaining system performance and reliability.

7. Non-Functional Requirements

7.1.Security Aspects

The system ensures security through the following measures:

- **Authentication & Authorization:**
  Only authorized users can access the system using secure login mechanisms.
- **Data Encryption:**
  Sensitive data such as passwords is stored in encrypted (hashed) form to prevent unauthorized access.
- **Secure Communication:**
  All communication between client and server is protected using secure protocols like HTTPS.
- **Access Control:**
  Role-based or user-based access control ensures that users can only access permitted resources.
- **Infrastructure Security:**
  Cloud resources (AWS EC2) are secured using SSH keys, firewalls, and proper configuration settings.

7.2.Performance Aspects

The system is designed to deliver fast and efficient performance under varying workloads.

- **Optimized Deployment:**
  CI/CD pipelines ensure quick build and deployment of applications, reducing downtime.
- **Scalability:**
  The system can scale infrastructure (e.g., EC2 instances) to handle increasing user demand.
- **Efficient Resource Utilization:**
  Docker containers ensure lightweight and efficient use of system resources.
- **Reduced Latency:**
  Caching mechanisms and optimized database queries help in faster response times.
- **High Availability:**
  The system ensures that the application remains accessible and performs reliably even under heavy usage.

8. References

The following resources were used for understanding and implementing the system:

- Terraform Documentation – https://developer.hashicorp.com/terraform/docs
- Docker Documentation – https://docs.docker.com/
- AWS EC2 Documentation – https://docs.aws.amazon.com/ec2/
- GitHub Actions Documentation – https://docs.github.com/en/actions
- Node.js Documentation – https://nodejs.org/
- Express.js Documentation – https://expressjs.com/